

Chapter 10 - Interfaces and structural typing

Object-oriented TypeScript, test first

Shapes

Add circle

Add rectangle

Add triangle

Clear



6 shapes · total area 29593.3

Shape	Area	Perimeter
 Circle	5026.5	251.3
 Rectangle	5400.0	300.0
 Triangle	3600.0	273.6

What you will learn

- `interface` and `implements`
- the compiler as your first red
- **structural typing**: shape, not name
- two interfaces, one object
- optional and `readonly` members
- `type` or `interface` ?
- interfaces vanish at run time
- object literals as test fakes

Why interfaces?

- drawing code wants: *for every shape, draw it*
- totals code wants: *for every shape, add its area*
- neither wants `if (circle) ... else if (rectangle) ...`
- neither should change when a `Hexagon` arrives

A **contract**: what every shape can do.

Java: `interface Shape` + `class Circle implements Shape`

Writing an interface

```
export interface Shape {  
  readonly name: string;  
  readonly colour: string;  
  area(): number;  
  perimeter(): number;  
  draw(pen: Pen): void;  
}
```

- members with types, no bodies
- **properties** too - not just methods (unlike Java)
- everything is public; no `public` keyword

Implementing it

```
export class Circle implements Shape {  
  public readonly name: string = "Circle";  
  
  constructor(  
    private readonly x: number,  
    private readonly y: number,  
    private readonly radius: number,  
    public readonly colour: string,  
  ) {}  
  
  public area(): number {  
    return Math.PI * this.radius * this.radius;  
  }  
  // ... perimeter(), draw(pen)  
}
```

The interface is the type

```
let shapes: Shape[] = [  
  new Circle(60, 60, 40, COLOURS[0]),  
  new Rectangle(180, 60, 90, 60, COLOURS[1]),  
  new Triangle(300, 60, 90, 80, COLOURS[2]),  
];  
  
export const totalArea = (shapes: Shape[]): number =>  
  shapes.reduce((total, shape) => total + shape.area(), 0);
```

- `totalArea` never asks which kind of shape
- a new `Hexagon` works with no change

Rectangle, test first: red 1

```
Deno.test("a 3 by 4 rectangle has an area of 12", () => {  
  const rectangle = new Rectangle(0, 0, 3, 4, "blue");  
  assertEquals(rectangle.area(), 12);  
});
```

```
TS2307 [ERROR]: Cannot find module 'src/Rectangle.ts'.
```

Red 2: the compiler's checklist

```
export class Rectangle implements Shape {  
  constructor(/* x, y, width, height, colour */) {}  
}
```

```
TS2420 [ERROR]: Class 'Rectangle' incorrectly  
  implements interface 'Shape'.  
  Type 'Rectangle' is missing the following  
  properties from type 'Shape':  
  name, area, perimeter, draw
```

`implements` turns the interface into a checklist.

Red 3, then green

Add every member, doing nothing (`return 0;`):

```
- 0  
+ 12
```

A real assertion failure. Then:

```
public area(): number {  
    return this.width * this.height;  
}
```

Green. Repeat for `perimeter` and `draw` .

How do you test drawing?

```
export interface Pen {  
  beginPath(): void;  
  arc(x: number, y: number, radius: number,  
    startAngle: number, endAngle: number): void;  
  rect(x: number, y: number,  
    width: number, height: number): void;  
  moveTo(x: number, y: number): void;  
  lineTo(x: number, y: number): void;  
  closePath(): void;  
  fill(): void;  
}
```

Our own interface: just what a shape needs.

The canvas fits the Pen

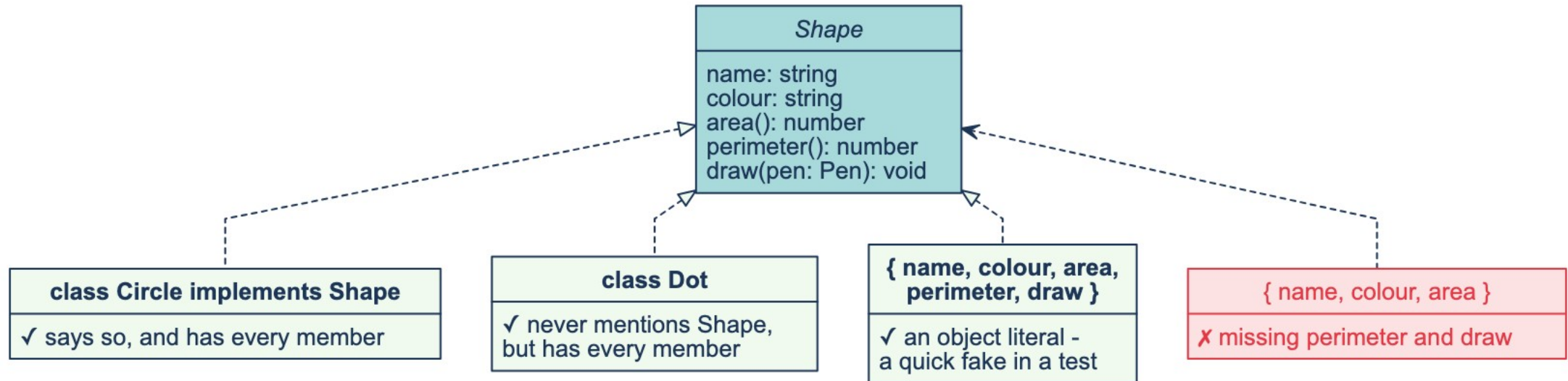
```
for (const shape of shapes) {  
  context.fillStyle = shape.colour;  
  // a CanvasRenderingContext2D, passed as a Pen  
  shape.draw(context);  
}
```

- the browser's canvas context never says `implements Pen`
- in Java: an error
- in TypeScript: fine – it **has** every method

Nominal vs structural

Java (nominal)	TypeScript (structural)
a <code>Shape</code> if it <i>says</i> <code>implements Shape</code>	a <code>Shape</code> if it <i>has</i> the members
the name matters	the shape matters
fakes need classes or Mockito	an object literal is a fake

Is it a Shape?



In Java only **Circle** would count: a Java class is a Shape only if it says `implements Shape`.

Why still write implements?

```
export class Dot {                // no implements
  public aera(): number { return 0; } // typo!
  // ...
}
const shapes: Shape[] = [new Dot()];
```

```
TS2741 [ERROR]: Property 'area' is missing in type
'Dot' but required in type 'Shape'.
```

Reported on the **array**, far away. With `implements`, it is on the class.

A fit that fails

```
export interface StrictPen {  
  fillStyle: string;  
  fill(): void;  
}
```

Types of property 'fillStyle' are incompatible.
Type 'string | CanvasGradient | CanvasPattern'
is not assignable to type 'string'.

Types are checked too. So `Pen` holds only methods.

Fakes: object literals

```
const fakeShape = (area: number): Shape => ({
  name: "fake",
  colour: "black",
  area: () => area,
  perimeter: () => 0,
  draw: () => {},
});

Deno.test("the total area adds up every shape's area", () => {
  assertEquals(
    totalArea([fakeShape(10), fakeShape(5), fakeShape(2.5)]),
    17.5);
});
```

A pen that records

```
const calls: string[] = [];  
const pen: Pen = {  
  beginPath: () => {  
    calls.push("beginPath");  
  },  
  rect: (x, y, width, height) => {  
    calls.push(`rect ${x} ${y} ${width} ${height}`);  
  },  
  // ... the other five  
};
```

```
new Rectangle(100, 50, 40, 20, "blue").draw(recorder.pen);  
assertEquals(recorder.calls,  
  ["beginPath", "rect 80 40 40 20", "fill"]);
```

Optional members

```
export interface Instrument {  
  readonly name: string;  
  readonly strings?: number;    // optional  
  play(): string;  
}  
  
export class Drum implements Instrument {  
  public readonly name: string = "Drum";  
  public play(): string {  
    return "Boom!";  
  }  
}
```

Optional means | undefined

```
instrument.strings + 1
```

TS18048 [ERROR]: 'instrument.strings' is possibly 'undefined'.

```
if (instrument.strings === undefined) {  
    return `${instrument.name} · no strings`;  
}  
return `${instrument.name} · ${instrument.strings} strings`;
```

The Drum surprise

- through `Instrument` : `drum.strings` is `undefined` - fine
- through `Drum` : an error

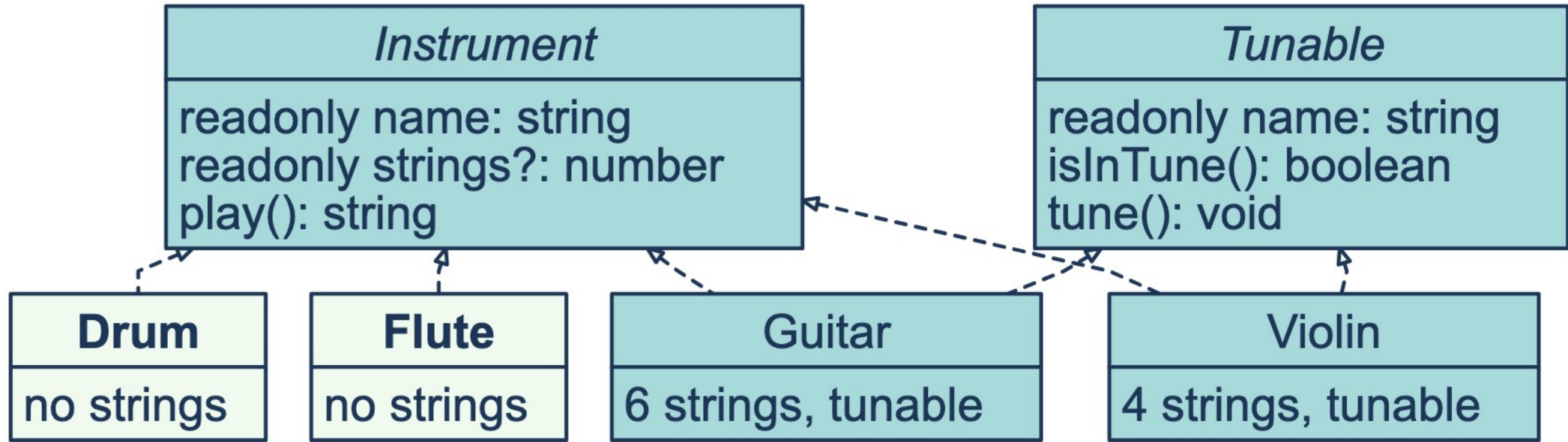
Property 'strings' does not exist on type 'Drum'.

```
const drum: Instrument = new Drum();  
assertEquals(drum.strings, undefined);
```

Two interfaces, one class

```
export interface Tunable {  
    readonly name: string;  
    isInTune(): boolean;  
    tune(): void;  
}  
  
export class Guitar implements Instrument, Tunable {  
    public readonly name: string = "Guitar";  
    public readonly strings: number = GUITAR_STRINGS;  
    constructor(private inTune: boolean = false) {}  
    // play(), isInTune(), tune()  
}
```

One guitar, two arrays



```
const band: Instrument[] = [guitar, violin, new Drum(), new Flute()];
const tunables: Tunable[] = [guitar, violin]; // the same two objects
```

Each interface is a **view**: `tuneAll` cannot play anything.

readonly - through a view

```
const shape: Shape = new Circle(0, 0, 1, "red");  
shape.colour = "blue";
```

TS2540 [ERROR]: Cannot assign to 'colour'
because it is a read-only property.

- a promise about the **view**, not the object
- a class may keep a changeable `public name: string`
- want it fixed everywhere? `readonly` in the class too

type or interface?

	interface	type
object shape	yes	yes
implements	yes	yes
build on another	extends	&
union "a" "b"	no	yes
function type	awkward	yes

Rule: interface for object shapes and what classes implement; type for the rest.

Interfaces vanish

```
// src/Circle.ts
var FULL_TURN = 2 * Math.PI;
var Circle = class {
  x;
  y;
  radius;
```

No `implements`, no `Shape.ts`, no `Pen.ts` in `dist/app.js`.

'Shape' only refers to a type, but is being used
as a value here. // value instanceof Shape

Sort anything

```
export interface Sortable {  
  label(): string;  
  sortKey(): number;    // smaller comes first  
}  
  
export class Mountain implements Sortable {  
  // ...  
  public sortKey(): number {  
    return -this.metres;    // tallest first  
  }  
}
```

Like Java's `Comparable`: each class picks its own order.

sortAll, test first with fakes

```
const item = (label: string, key: number): Sortable => ({
  label: () => label,
  sortKey: () => key,
});

assertEquals(labels(sortAll(
  [item("c", 3), item("a", 1), item("b", 2)])),
  ["a", "b", "c"]);
```

Red with `=> items`, then green:

```
items.toSorted((a, b) => a.sortKey() - b.sortKey())
```

Java and TypeScript

Java	TypeScript
<code>double area();</code>	<code>area(): number;</code>
methods and constants only	properties too
must say <code>implements</code>	anything that fits
fakes: classes, Mockito	object literals
<code>instanceof Shape</code>	an error: interfaces vanish
<code>Comparable<T></code>	your own interface; generics in Book 2

Summary and challenges

- interfaces list members; `implements` checks them
- structural: shape, not name - still write `implements`
- object literals make fakes; small interfaces fake easily
- `?` means `| undefined`; `readonly` guards a view
- interfaces are compile-time only

Challenges: 1 Square · 2 Cities · 3 Biggest shape ·
4 SoundMaker · 5 Ties · 6 Click to choose

Next: Chapter 11 - Inheritance